# REVIEW OF QUANTUM GATES AND CIRCUITRY

## A Summer Internship Report

VISHWAJEET KUMAR SINGH

Shri Mata Vaishno Devi University, Katra, Jammu & Kashmir

Under the

Supervision of Dr.

Rakesh Ranjan
Associate Professor, ECE Department, NIT Patna

# CERTIFICATE

This is to certify that Vishwajeet Kumar Singh (SMVDU) has successfully completed the summer internship on Review of Quantum Gates and Circuitry under the guidance of Dr. Rakesh Ranjan.

Supervisor's Signature:_____

Supervisor's Name: Dr. Rakesh Ranjan

# DECLARATION

I Vishwajeet Kumar Singh (SMVDU) , hereby declare that I have completed my summer internship on the subject 'Review of Quantum Gates and Circuitry' under the guidance of Dr. Rakesh Ranjan (Associate Professor, NIT PATNA) from National Institute of Technology Patna in the duration 2nd June – 3th July.

Signature:_____

# ACKNOWLEDGMENT

I would like to express my deepest gratitude to the individuals and the institution whose support and guidance have been invaluable to the successful completion of this internship.

First and foremost, I am profoundly grateful to the National Institute of Technology, Patna, for providing me with the opportunity to undertake this internship. The state-of-the-art facilities and a conducive research environment have been instrumental in the successful execution of this work. I extend my sincere thanks to the entire administrative and technical staff for their continuous support and assistance.

I am especially indebted to my esteemed supervisor and mentor, Dr. Rakesh Ranjan, Associate Professor, ECE Department, NIT Patna, whose exceptional guidance, insightful feedback, and unwavering support have been crucial throughout the course of this project. His profound expertise and meticulous attention to detail have significantly shaped the direction and depth of this research. I am deeply thankful for his patience, encouragement, and mentorship, which have been a source of inspiration and motivation.

Furthermore, I am grateful to the individuals who participated in this study, providing essential data and insights without which this research would not have been possible. Their cooperation and willingness to share their experiences have been valuable.

To my family and friends, I extend my deepest appreciation for their unwavering support and understanding throughout this journey. Their encouragement and belief in my abilities have been a constant source of motivation and strength, enabling me to persevere through the most challenging phases of this internship.

Finally, I wish to express my profound appreciation to all those who have directly or indirectly supported me in this endeavor. Your contributions, whether big or small, have been invaluable, and I am sincerely grateful for your support.

This acknowledgment, though detailed, may still fall short of conveying the full extent of my gratitude. Nonetheless, I hope it serves as a testament to the collective effort and collaboration that have culminated in the successful completion of this project.

(Vishwajeet Kumar Singh)

# TABLE OF CONTENTS

# 1. Introduction to Quantum Basics

## QUBITS (QUANTUM BITS)

In classical computing (the kind of computers we use every day), the basic unit of information is a bit. A bit can only have one of two values:
**0** or **1**
These values represent two different states like ON/OFF, TRUE/FALSE, or YES/NO.
But in quantum computing, we use something called a **qubit** (short for quantum bit).

### ❖ What is a Qubit?

**A** qubit **is the basic unit of information in a** quantum computer**. Unlike a regular bit, a qubit can be in:**

- **0**,
- **1**, **or**
- **both 0 and 1 at the same time**!

This "both at the same time" idea is called **superposition**.

## INFROMATION THEORY

Quantum computation is an entirely new way of **information processing**. For those new to the subject, we begin with a simple and brief review of how information is stored and used in computers. The most **basic piece of information is called a *bit*.**
**Information** is the **meaning** or **content** of the message.

**Example :**
In the message *"I will be late"*, the **information** is that someone is not coming on time.

So:

- **Message** = the thing that is sent.
- **Information** = the useful content inside the message.

### ❖ How much information is actually contained in that message?

$$m = 2^n$$

where, ***m is the given message***

This is the foundation of how quantum computers process and store information differently.

## STATE AND REPRESENTATION OF QUBITS

### ❖ State of Qubits

In quantum theory an object enclosed using the notation | ⟩ can be called a *state*, a *vector*, or a *ket*.

A qubit can exist in the state |0⟩ or the state |1⟩, but it can also exist in we call a *superposition* state.

If we label this state |$\psi$⟩, a **superposition state is written as**:

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$

***α*, *β* are complex numbers** i.e. the numbers are in form of $z = x + iy$, where $i = \sqrt{-1}$

✓ **NOTE**

I. $|\alpha|^2$: Tells us the probability of finding |$\psi$⟩ in state |0⟩.
II. $|\beta|^2$: Tells us the probability of finding |$\psi$⟩ in state |1⟩.
III. $|\alpha|^2 + |\beta|^2 = 1$

### ❖ Normalized Qubit

$$\sum_{i=1}^{N} p_i = p_1 + p_2 + \cdots + p_N = 1$$

When this condition is satisfied for the squares of the coefficients of a qubit, we say that the qubit is *normalized*.

✓ **NOTE**

$$|\alpha|^2 = (\alpha)(\alpha^*)$$
$$|\beta|^2 = (\beta)(\beta^*)$$

where ***α* is the complex conjugate*** of *α* and ***β* is the complex conjugate*** of *β*. We recall that to form the complex conjugate of ***z = x + iy***, we let $i \rightarrow -i$.

# 2. Probability Basics

## ENTROPY AND SHANNON'S THEORY

***Entropy***, a quantity that can be said to be a measure of disorder in Physics. Information is certainly the opposite of disorder, Entropy can be used to characterize the information content in a signal.

The **Hartley method** gives us a basic characterization of information content in a signal. But another **scientist named Claude Shannon** showed that we can take things a step further and get a **more accurate estimation of the information content in a signal.**

### ❖ According to Shannon:

- If a message has a very **high probability of occurrence**, then we don't gain all that much **(low content) new information** when we come across it.
- If a message has a **low probability of occurrence**, when we are made of aware of it, we **gain a significant amount of information**.
- If we denote the **information content of a message by *I*** , and the **probability of its occurrence by *p***, then:

$$I = -\log_2 p$$

### ❖ SHANNON ENTROPY

*Let* X *be a random variable characterized by a probability distribution* p*, and suppose that it can assume one of the values x 1, x 2, . . . , x n with probabilities p1, p2, . . . , pn.*
Probabilities satisfy $0 \leq pi \leq 1$ and $\sum_i p_i = 1.$

The ***Shannon entropy*** of *X* is defined as:

$$H(X) = -\sum_i p_i \log_2 p_i$$

### ❖ Summary of Shannon's Information Theory

- Decrease uncertainty ⇒ Increase information
- Increase uncertainty ⇒ Increase entropy

## PROBABILITY IN QUANTUM

Probability is heavily used in quantum to predict the possible results of measurement. The probability of an **event that is *impossible* is 0**. The probability of an event that is ***certain to happen* is 1.**

All other probabilities fall within this range:

$$0 \leqslant p_i \leqslant 1$$

The probability of an event is simply the relative frequency of its occurrence.

Suppose that there are $n$ total events, the $j^{th}$ event occurs $n_j$ times, and we have,

$$\sum_{j=1}^{\infty} nj = n$$

Then, the **probability that the $j_{th}$ event occurs is:**

$$p_j = \frac{n_j}{n}$$

Sum of all probabilities is always equal to 1.

$$\sum_{j=1}^{\infty} p_j = \sum_{j=1}^{\infty} \frac{n_j}{n} = \frac{1}{n} \sum_{j=1}^{\infty} n_j = \frac{n}{n} = 1$$

### ❖ Variance

The variance of a given distribution is equal to:

$$\langle (\Delta j)^2 \rangle = \langle j^2 \rangle - \langle j \rangle^2$$

Where,

$$\langle j \rangle = \sum_{j=1}^{\infty} \frac{j n_j}{n} = \sum_{j=1}^{\infty} j p_j$$

# 3. VECTOR SPACES

## INTRODUCTION

The arena in which quantum computation takes place is a mathematical abstraction called a *vector space*. It turns out that quantum states behave mathematically in an analogous way to physical vectors.

A vector space $V$ is a nonempty set with elements $u, v$ called *vectors* for which the following two operations are defined:

1. **Vector addition**: An operation that assigns the sum $w = u + v$, which is also element of $V$ ; and $w$ is another vector belonging to the same space.
2. **Scalar multiplication**: Defines multiplication of a vector by a number $\alpha$ such that the vector $\alpha u \in V$.

### ❖ Vector Space of "*n*-tuples"

One particular vector space that is important in quantum computation is the vector space $C^n$, which is the vector space of "$n$-tuples" of complex numbers.

- ***n*-tuples:** Referring to an ordered collection of numbers.

We label the elements of $C^n$ by $|a\rangle$ , $|b\rangle$ , $|c\rangle$

$$|a\rangle = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix}$$

### ❖ Basis and Dimension

When a set of vectors is linearly independent and they span the space, the set is known as a ***basis***.

The ***dimension*** of a vector space $V$ is equal to the number of elements in the basis set.

Basis set for $C^2$, with the qubit states $|0\rangle$ and $|1\rangle$:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

## INNER PRODUCT

An **inner product** is a way to **multiply two vectors** to get a **scalar** (a single number), often used to measure:

- **Angle** or **closeness** between vectors.
- Whether two vectors are **orthogonal** (perpendicular).
- The **length (norm)** of a vector.

In quantum computing, we're dealing with **complex vector spaces** like $C^n$, so we use the **complex inner product**, sometimes called the **Hermitian inner product**.

### ✓ NOTE

I. The inner product will take two vectors from $C^n$ and map them to a ***complex*** number.
II. The inner product between two vectors |u⟩, |v⟩ with the notation $\langle u|v\rangle$ .
III. If the inner product between two vectors is zero,

$$\langle u|v\rangle = 0$$ $|u\rangle, |v\rangle$ are ***orthogonal*** to one another.

### ❖ Hermitian Conjugate

To compute the inner product between two vectors, we must calculate the Hermitian conjugate of a vector.

$$(|u\rangle)^\dagger = \langle u|$$

- $\langle u|$ is sometimes called the *dual vector* or *bra* corresponding to $|u\rangle$ .

**Hermitian conjugate** is computed in two steps:

1. Write the components of the vector as a row of numbers.
2. Take the complex conjugate of each element and arrange them in a row vector.

$$\begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix}^\dagger = \begin{pmatrix} a_1^* & a_2^* & \dots & a_n^* \end{pmatrix}$$

where, a* is the **conjugate of complex numbers**.

## ❖ Calculation of Inner Product

$$\langle a|b\rangle = \begin{pmatrix} a_1^* & a_2^* & \dots & a_n^* \end{pmatrix} \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix} = a_1^* b_1 + a_2^* b_2 + \cdots + a_n^* b_n = \sum_{i=1}^{n} a_i^* b_i$$

## ❖ Properties

1. **Linearity in second argument**:

$$\langle u|\alpha v + \beta w\rangle = \alpha\langle u|v\rangle + \beta\langle u|w\rangle$$
$$\langle \alpha u + \beta v|w\rangle = \alpha^*\langle u|w\rangle + \beta^*\langle v|w\rangle$$

2. **Conjugate symmetry**:

$$\langle u|v\rangle^* = \langle v|u\rangle$$

## ❖ Uses of Inner Product

In quantum computing, it's used to compute:

- Probabilities
- Norms (lengths)
- Orthogonality (two states are 90 degrees to each other)
- Gate/unitary behavior

# 4. MATRICES IN QUANTUM

## INTRODUCTION

In quantum computing:

- **Quantum states** are vectors
- **Quantum gates/operations** are **matrices** that act on those vectors.
- These matrices are **unitary** (they preserve the norm and inner product).

## ❖ Quantum Gates as Matrices

Quantum gates are **unitary matrices** U that transform quantum states:

| GATE | MATRIX | EFFECT |
|---|---|---|
| **1. I (Identity)** | $\begin{bmatrix}1 & 0\\0 & 1\end{bmatrix}$ | Does nothing |
| **2. X (NOT / Pauli-X)** | $\begin{bmatrix}0 & 1\\1 & 0\end{bmatrix}$ | Flips qubit |
| **3. Y (Pauli-Y)** | $\begin{bmatrix}0 & -i\\i & 0\end{bmatrix}$ | Rotation with phase |
| **4. Z (Pauli-Z)** | $\begin{bmatrix}1 & 0\\0 & -1\end{bmatrix}$ | Adds a phase to $\|1\rangle$ |
| **5. H (Hadamard)** | $\frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\1 & -1\end{bmatrix}$ | Creates superposition |
| **6. CNOT** | $\begin{bmatrix}1 & 0 & 0 & 0\\0 & 1 & 0 & 0\\0 & 0 & 0 & 1\\0 & 0 & 1 & 0\end{bmatrix}$ | Two qubit gate: flips target if control is 1 |

## ❖ Uses of Matrix

1. Used to apply gates to quantum states.
2. Used to **combine gates** or **combine qubit states**.
3. Projection matrices determine probabilities.

## Outer Product

- The **outer product** of two vectors |u⟩ and |v⟩ is written as: **|u⟩⟨v|**
- This operation produces a **matrix**, not a scalar.

### ❖ Calculation

The outer product |□⟩ ⟨ □| is calculated in the following way:

$$|\psi\rangle\langle\phi| = \begin{pmatrix} a \\ b \end{pmatrix} \begin{pmatrix} c^* & d^* \end{pmatrix} = \begin{pmatrix} ac^* & ad^* \\ bc^* & bd^* \end{pmatrix}$$

$$|\phi\rangle\langle\psi| = \begin{pmatrix} c \\ d \end{pmatrix} \begin{pmatrix} a^* & b^* \end{pmatrix} = \begin{pmatrix} ca^* & cb^* \\ da^* & db^* \end{pmatrix}$$

### ❖ Uses of Outer Product

- *Projection operators*: Used in measurement to project a state onto a basis vector.
- *Density matrices*: A **density matrix** ρ is a way of describing a **quantum state.**
- *State reconstruction*: The outer product gives a matrix that encodes full state information.
- *Building operators*: You can construct custom quantum operators or observables using outer products of basis vectors.

### ❖ Inner vs Outer Product

| Feature | Inner Product | Outer Product |
|---|---|---|
| **Input** | Two vectors | Two vectors |
| **Output** | A scalar (number) | A matrix |
| **Purpose** | Measures overlap (similarity, angle) | Constructs matrices (projections, density) |
| **Shape** | 1×1 scalar | $n \times n$ matrix (if vectors of size $n$) |
| **Use in Quantum** | Probabilities, normalization, orthogonality | projectors, density matrices, operators |

# 5. QUANTUM OPERATORS AND GATES

An *operator* is a mathematical rule that can be applied to a function to transform it into another function. Operators are often indicated by placing a caret above the symbol used to denote the operator.

They act on quantum states (vectors) to **transform, measure, or describe** them. They are the core of every quantum algorithm.

## UNITARY OPERATORS

### ❖ Single Qubit Gate Operators

| GATE | MATRIX | EFFECT |
|---|---|---|
| **1. I (Identity)** | $\begin{bmatrix}1 & 0\\0 & 1\end{bmatrix}$ | Does nothing |
| **2. X (NOT / Pauli-X)** | $\begin{bmatrix}0 & 1\\1 & 0\end{bmatrix}$ | Flips qubit |
| **3. Y (Pauli-Y)** | $\begin{bmatrix}0 & -i\\i & 0\end{bmatrix}$ | Rotation with phase |
| **4. Z (Pauli-Z)** | $\begin{bmatrix}1 & 0\\0 & -1\end{bmatrix}$ | Adds a phase to $\|1\rangle$ |
| **5. H (Hadamard)** | $\frac{1}{\sqrt{2}}\begin{bmatrix}1 & 1\\1 & -1\end{bmatrix}$ | Creates superposition |
| **6. S (Phase)** | $\begin{bmatrix}1 & 0\\0 & i\end{bmatrix}$ | Phase shift of $\pi/2$ |
| **7. T ($\pi/8$ gate)** | $\begin{bmatrix}1 & 0\\0 & e^{-i\pi/4}\end{bmatrix}$ | Phase shift of $\pi/4$ |
| 8. **Rx(θ)** | $\cos(\theta/2)\mathrm{I} - i\sin(\theta/2)\mathrm{X}$ | Rotation around X-axis |
| **9. Ry(θ)** | $\cos(\theta/2)\mathrm{I} - i\sin(\theta/2)\mathrm{Y}$ | Rotation around Y-axis |

## ❖ Multi Qubit Gate

| Gate/Operator | Description | Matrix Size |
|---|---|---|
| **CNOT (CX)** | Flips target qubit if control is 1 | 4 x 4 |
| **Toffoli (CCX)** | Controlled-CNOT (2 controls) | 8 x 8 |
| **SWAP** | Swaps two qubits | 4 x 4 |
| **Controlled-Z (CZ)** | Applies Z to target if control is 1 | 4 x 4 |
| **Controlled-U** | Applies U to target if control is 1 | varies |

## HERMITIAN, NORMAL AND UNITARY OPERATORS

| Properties | Normal | Hermitian | Unitary |
|---|---|---|---|
| **Definition** | $A^\dagger A=AA^\dagger$ | $A=A^\dagger$ | $A^\dagger A=AA^\dagger=I$ |
| **Diagonalizable?** | Yes (by unitary matrix) | Yes (with real eigen values) | Yes (with complex unit modulus eigenvalues |
| **Eigenvalues** | May be complex | Real | On unit circle (modulus = 1) |
| **Importance in Quantum** | Generalized structure | Observable | Quantum gates (evolution) |

Here $A^\dagger$ is known as **Hermitian Adjoint.**

✓ **NOTE**

- To calculate Hermitian Adjoint follow steps:

1. To take complex conjugate ($\overline{A}$) of matrix A.

   $A = \begin{bmatrix} 1 & i \\ 2 & -3i \end{bmatrix}$ then $(\overline{A}) = \begin{bmatrix} 1 & -i \\ 2 & 3i \end{bmatrix}$

2. Take transpose of ($\overline{A}$)

   $A^\dagger = (\overline{A})^T = \begin{bmatrix} 1 & 2 \\ -i & 3i \end{bmatrix}$

# 6. QUANTUM CIRCUITS

## INTRODUCTION

A **quantum circuit** is like a recipe or a set of instructions that tells a **quantum computer** what to do.

- In regular (classical) computers, we use **logic gates** like AND, OR, and NOT to process bits (which are 0 or 1).
- In quantum computers, we use **quantum gates** to process **qubits**.

**Qubits** are special because they can be 0, 1, or both at the same time (thanks to something called **superposition**).

A **quantum circuit** is a combination of quantum gates connected in a certain way, which tells the computer how to manipulate qubits step by step.

### ❖ Parts of Quantum Circuit

1. **Qubits**: These are the "wires" in a quantum circuit – like the data holders.
2. **Quantum Gates**: These are the actions or operations (like rotating, flipping, or entangling qubits).
3. **Measurement**: At the end, we measure the qubits to get a classical result (0s or 1s).

### ❖ Common Quantum Gates

1. **Hadamard Gate (H)**: Puts qubits into superposition (makes them be 0 and 1 at once).
2. **Pauli Gates (X, Y, Z)**: Basic operations like flipping the qubit (X is like a NOT gate).
3. **CNOT Gate (Controlled-NOT)**: Connects two qubits and creates **entanglement** (when qubits affect each other).

## ROLE OF CODING IN QUANTUM CIRCUIT

In quantum computing, we **use code to create and run quantum circuits**.
You don't build circuits with physical wires — instead, you **write programs** that tell a quantum computer what circuit to build and how to run it.

So, **coding** helps you:

1. **Create** quantum circuits (add qubits and quantum gates).

2. **Simulate** the circuit on your regular computer (before running on a real quantum computer).
3. **Run** the circuit on actual quantum computers (like IBM's, Google's, etc.).
4. **Analyze** the output or results.

## ❖ Why Python Language is Used

1. **Easy to learn and use:** Python code is simple and readable, even for beginners.
2. **Strong Quantum Libraries:** Python has powerful **quantum computing libraries** made by companies and researchers, such as:

    - Qiskit (from IBM)
    - Criq (from Google)
    - Braket (from Amazon)

## QISKIT LIBRARY

**Qiskit** is an open-source Python library developed by IBM for working with quantum computers. It allows users to **create, simulate, and run quantum circuits** on both simulators and real quantum hardware. Qiskit makes it easy to explore **quantum algorithms**, **visualize circuits**, and analyze results. It's widely used in quantum research, education, and development.

## ❖ Programming Using Qiskit in Python

- Basic Circuit Design

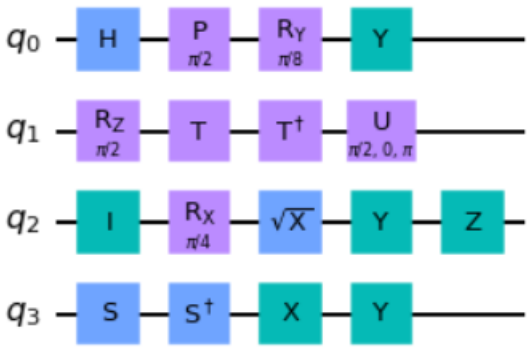
***1. Program***

```
import math

qc = QuantumCircuit(4)
qc.h(0)
qc.id(2)
qc.i(2)
qc.p(math.pi/2,0)
qc.rx(math.pi/4,2)
qc.ry(math.pi/8,0)
qc.rz(math.pi/2,1)
qc.s(3)
qc.sdg(3)
qc.sx(2)
qc.t(1)
qc.tdg(1)
qc.u(math.pi/2,0,math.pi,1)
qc.x(3)
qc.y([0,2,3])
qc.z(2)
```

***Output*** `<qiskit.circuit.instructionset.InstructionSet at 0x7f262034e100>`

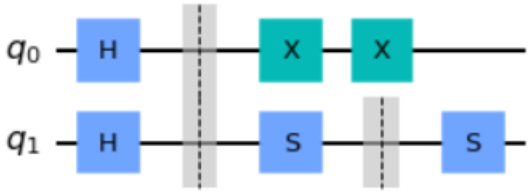
## 2. *Program* *Output*

```
qc = QuantumCircuit(4)
qc.h(0)
qc.id(2)
qc.p(math.pi/2,0)
qc.rx(math.pi/4,2)
qc.ry(math.pi/8,0)
qc.rz(math.pi/2,1)
qc.s(3)
qc.sdg(3)
qc.sx(2)
qc.t(1)
qc.tdg(1)
qc.u(math.pi/2,0,math.pi,1)
qc.x(3)
qc.y([0,2,3])
qc.z(2)
qc.draw('mpl')
```



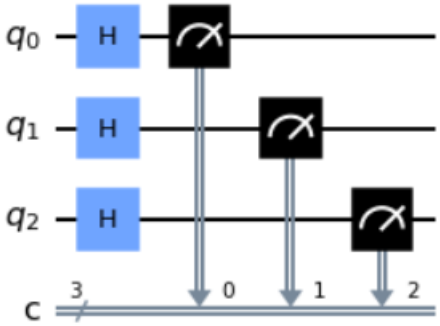

## 3. *Program*

```
qc = QuantumCircuit(2)
qc.h([0,1])
qc.barrier()
qc.x(0)
qc.x(0)
qc.s(1)
qc.barrier([1])
qc.s(1)
qc.draw('mpl')
```




## 4. *Program*

```
qc = QuantumCircuit(3, 3)
qc.h([0,1,2])
qc.measure([0,1,2], [0,1,2])
qc.draw('mpl')
```



- Analyze the circuit

1. ***Program***

```
from qiskit import QuantumCircuit,BasicAer,transpile

qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)

backend = BasicAer.get_backend("unitary_simulator")
tqc = transpile(qc, backend)
job = backend.run(tqc)
result = job.result()
unitary = result.get_unitary(tqc, 4)
print(unitary)
```

***Output***

```
[[ 0.7071+0.j  0.7071-0.j  0.    +0.j  0.    +0.j]
 [ 0.    +0.j  0.    +0.j  0.7071+0.j -0.7071+0.j]
 [ 0.    +0.j  0.    +0.j  0.7071+0.j  0.7071-0.j]
 [ 0.7071+0.j -0.7071+0.j  0.    +0.j  0.    +0.j]]
```

2. ***Program***

```
from qiskit import QuantumCircuit,BasicAer,transpile

qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)

backend = BasicAer.get_backend("statevector_simulator")
tqc = transpile(qc, backend)
job = backend.run(tqc)
result = job.result()
statevector = result.get_statevector(tqc, 4)
print(statevector)
```

***Output***

```
[0.7071+0.j 0.    +0.j 0.    +0.j 0.7071+0.j]
```

3. ***Program***

```
from qiskit import QuantumCircuit,Aer,transpile

qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)

backend = Aer.get_backend("aer_simulator")
qc.save_unitary()
tqc = transpile(qc, backend)
job = backend.run(tqc)
result = job.result()
unitary = result.get_unitary(qc, 4)
print(unitary)
```

***Output***

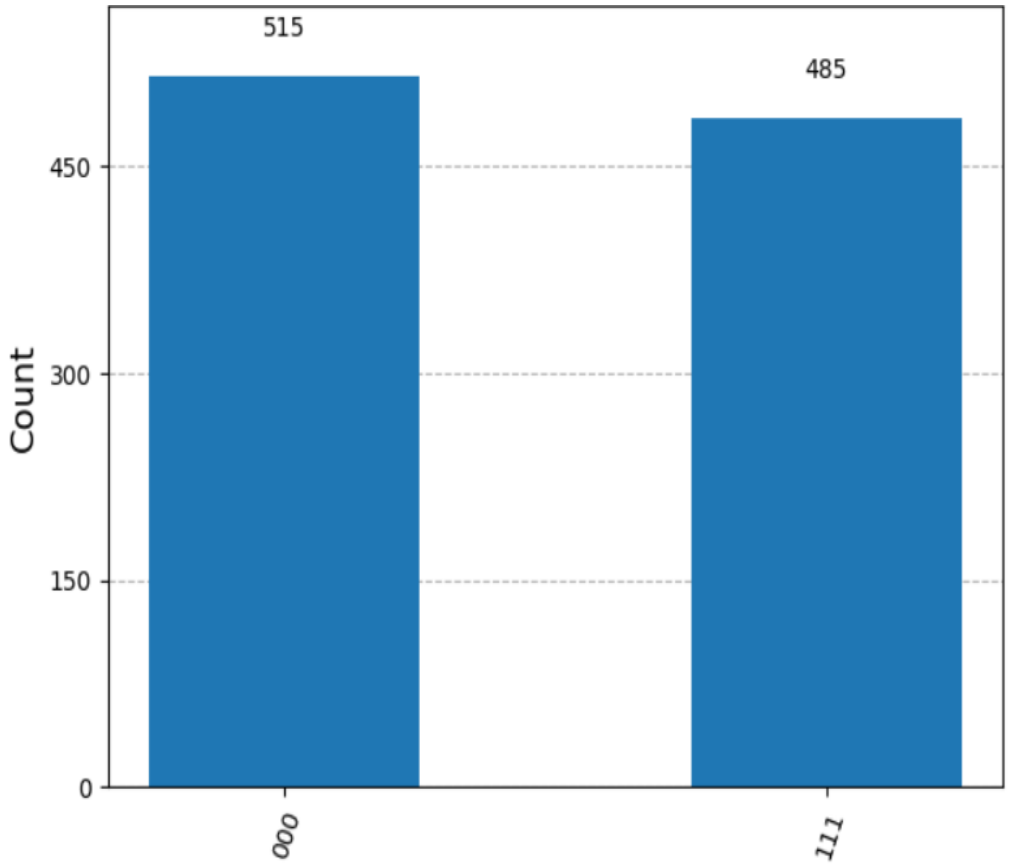
```
Operator([[ 0.7071+0.j,  0.7071-0.j,  0.    +0.j,  0.    +0.j],
          [ 0.    +0.j,  0.    +0.j,  0.7071+0.j, -0.7071+0.j],
          [ 0.    +0.j,  0.    +0.j,  0.7071+0.j,  0.7071-0.j],
          [ 0.7071+0.j, -0.7071+0.j,  0.    +0.j,  0.    +0.j]],
         input_dims=(2, 2), output_dims=(2, 2))
```

- Visualizing the result

1. ***Program***

```
!pip install qiskit[all]
!pip install qiskit-aer
from qiskit import QuantumCircuit
from qiskit.quantum_info import DensityMatrix, \
                                        Operator
from qiskit import QuantumCircuit, transpile
from qiskit_aer import Aer
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt
from qiskit.visualization import plot_state_qsphere
from math import pi

qc = QuantumCircuit(3)
qc.h(0)
qc.cx(0,1)
qc.cx(0,2)

qc.measure_all()
qc.draw()

backend = Aer.get_backend("aer_simulator")
tqc = transpile(qc, backend)
job = backend.run(tqc, shots=1000)
result = job.result()
counts = result.get_counts(tqc)

plot_histogram(counts)
```

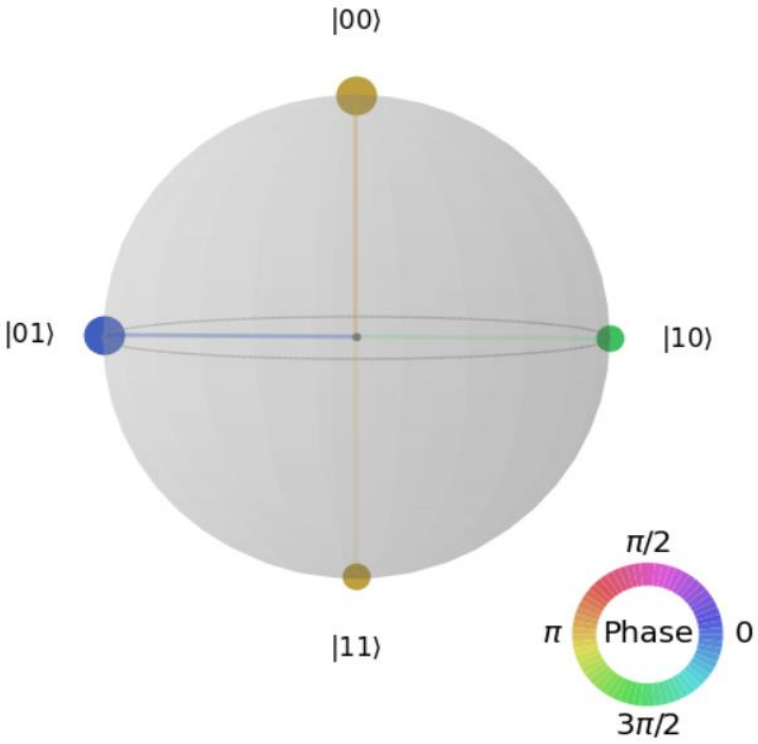
***Output***




## 2. *Program*

```
backend = Aer.get_backend("aer_simulator_statevector")

qc = QuantumCircuit(2)
qc.rx(pi, 0)
qc.ry(pi/8, 1)
qc.swap(0, 1)
qc.h(0)
qc.cp(pi/2,0, 1)
#qc.cp(pi/4, 0, 2)
qc.h(1)
qc.cp(pi/2, 0, 1)
# qc.h(2)
qc.save_statevector()

tqc = transpile(qc, backend)
job = backend.run(tqc)
result = job.result()
statevector = result.get_statevector()

plot_state_qsphere(statevector)
```

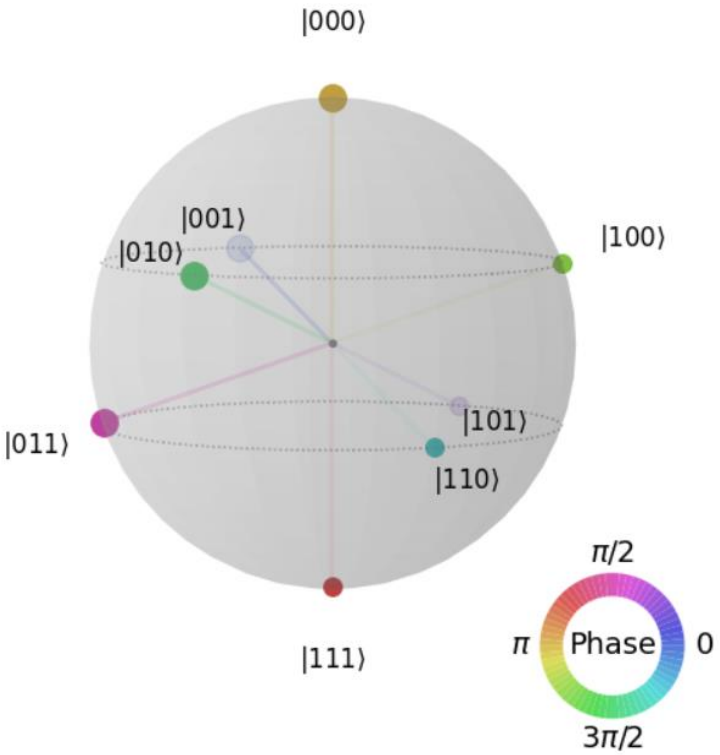
***Output***




### 3. *Program*

```
backend = Aer.get_backend("aer_simulator_statevector")

backend = Aer.get_backend("aer_simulator_statevector")

qc = QuantumCircuit(3)
qc.rx(pi, 0)
qc.ry(pi/8, 2)
qc.swap(0, 2)
qc.h(0)
qc.cp(pi/2,0, 1)
qc.cp(pi/4, 0, 2)
qc.h(1)
qc.cp(pi/2, 1, 2)
qc.h(2)
qc.save_statevector()

tqc = transpile(qc, backend)
job = backend.run(tqc)
result = job.result()
statevector = result.get_statevector()

plot_state_qsphere(statevector)
```

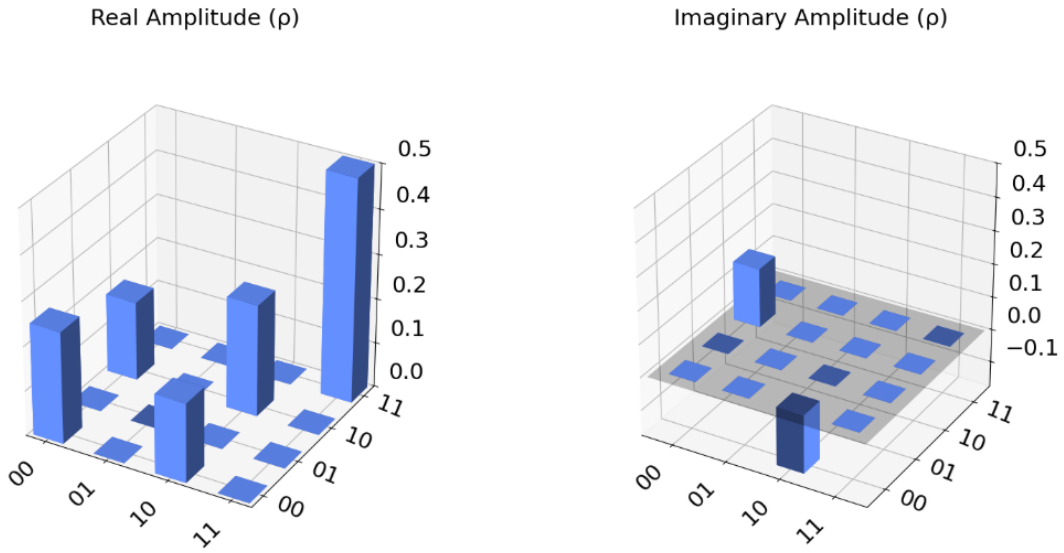
***Output***




## 4. *Program*

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import DensityMatrix, \
                                        Operator
from qiskit.visualization import plot_state_city
from qiskit import QuantumCircuit, transpile
from qiskit_aer import Aer
from qiskit.visualization import plot_histogram
import matplotlib.pyplot as plt
from qiskit.visualization import plot_state_qsphere
from math import pi
dens_mat = 0.5*DensityMatrix.from_label('11') + \
  0.5*DensityMatrix.from_label('+0')
tt_op = Operator.from_label('TT')
dens_mat = dens_mat.evolve(tt_op)

plot_state_city(dens_mat)
```

***Output***


Real Amplitude (ρ)
Imaginary Amplitude (ρ)
0.5
0.4
0.3
0.2
0.1
0.0
−0.1
00
01
10
11

# REFERENCES


1. Book: Quantum Computing Explained (Author David McMahon)

2. https://github.com/qiskit-community/qiskit-pocket-guide/blob/main/chapter03_Visualizing_Quantum_Measurements_and_States/chapter03-2_Visualizing_Quantum_States.ipynb

3. https://github.com/qiskit-community/qiskit-pocket-guide/tree/main/chapter01_Quantum_Circuits_and_Operations

4. https://github.com/qiskit-community/qiskit-pocket-guide/tree/main/chapter02_Running_Quantum_Circuits

5. https://colab.research.google.com/drive/1hWYCbVPe7oDhF-kOkHxghM5i27PsU8Ce#scrollTo=R43aJcjGMPum